Лекция 6
Управляющие конструкции языка PHP

Условные операторы

Группы управляющих конструкций PHP, которые будут рассмотрены:

Условные операторы:

```
if
else
elseif
```

Конструкции выбора:

switch-case

Циклы:

```
while
do-while
for
Foreach
```

Другие:

```
break
Continue
return
```

Условные операторы. Конструкция if

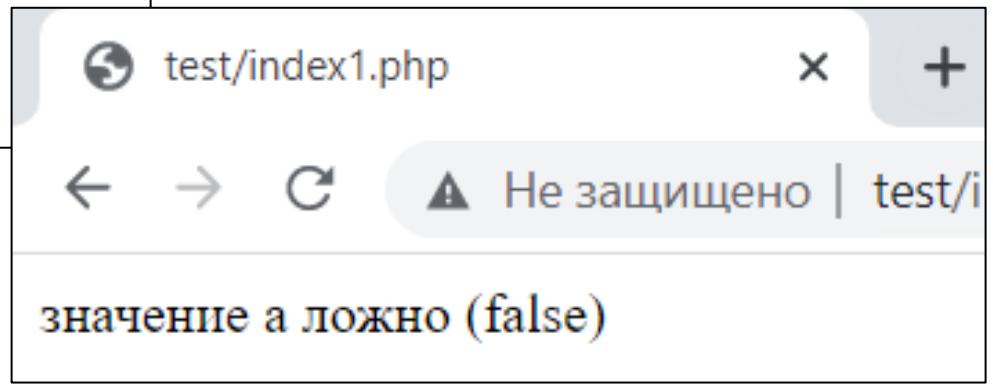
```
<?php
if (логическое выражение) оператор;
?>
```

Согласно выражениям PHP, конструкция **if** содержит логическое выражение. Если логическое выражение истинно (**true**), то оператор, следующий за конструкцией **if** будет исполнен, а если логическое выражение ложно (**false**), то следующий за **if** оператор исполнен не будет.

Условные операторы. Конструкция if

```
<?php
if ($a > $b) echo "значение a больше, чем b";
?>
```

```
<?php
$a = false;
if (!$a) echo "значение a ложно (false)";
?>
```

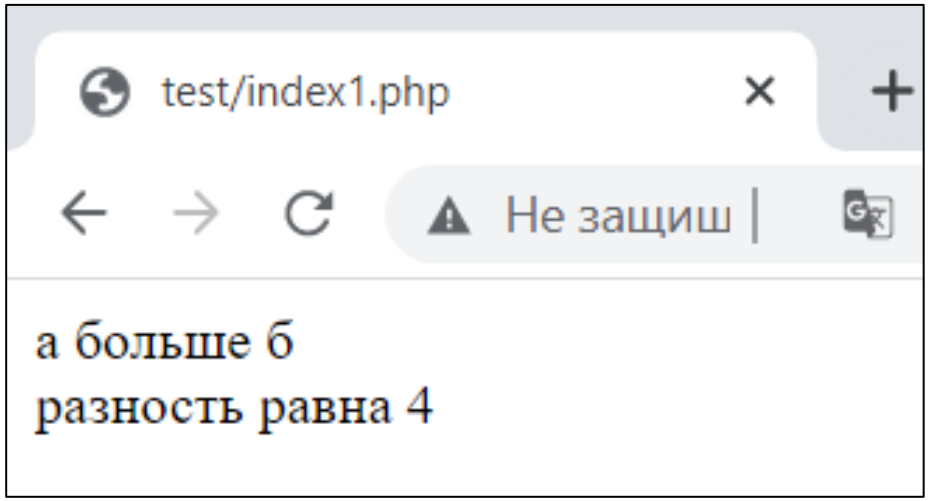


Условные операторы. Конструкция if

Часто вам будет необходим блок операторов, который будет выполняться при определенном условном критерии, тогда эти операторы необходимо поместить в фигурные скобки **{...}**

Пример:

```php
<?php
    $a = 5;
    $b = 1;
    if ($a > $b) {
        echo "a больше b";
        $c = $a - $b;
        echo "<br>", "разность равна $c";
    }
?>
```



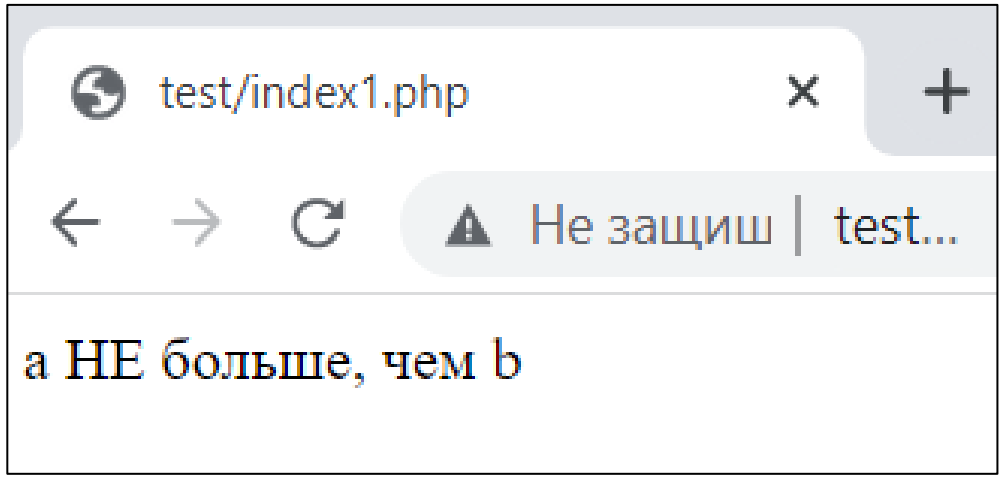
Условные операторы. Конструкция if-else

Часто возникает потребность исполнения операторов не только в теле конструкции **if**, если выполнено какое-либо условие конструкции **if**, но и в случае, если условие конструкции **if** не выполнено. В данной ситуации нельзя обойтись без конструкции **else**. В целом, такая конструкция будет называться конструкцией **if-else**.

```
if (логическое_выражение)
   инструкция_1;
else
   инструкция_2;
```

Условные операторы. Конструкция if-else

```php
<?php
    $a = 5;
    $b = 6;
    if ($a > $b) {
        echo "a больше, чем b";
    } else {
        echo "a НЕ больше, чем b";
    }
?>
```

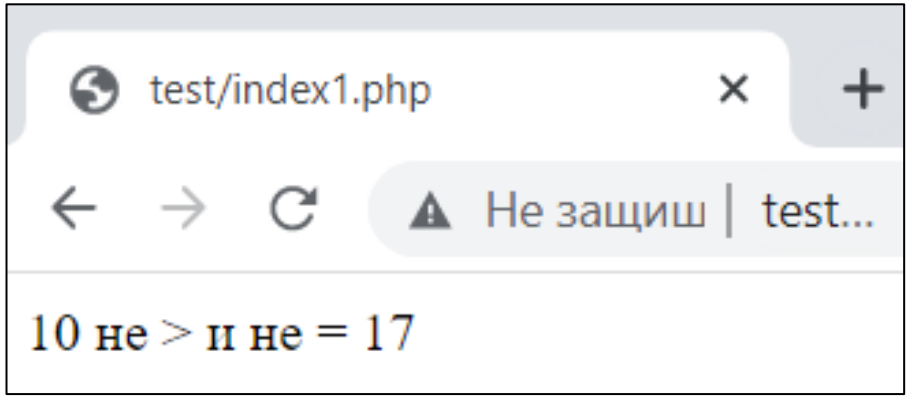


Условные операторы. Конструкция if-else (альтернативный синтаксис)

```
if (логическое_выражение):
   команды;
else:
   иначе_команды;
endif
```

Условные операторы. Конструкция if-else (альтернативный синтаксис)

```php
<?php
    $x = 10;
    $y = 17;
    if($x > $y):
        echo $x." больше, чем ".$y;
    elseif($x == $y):
        echo $x." равно ".$y;
    else:
        echo $x." не > и не = ".$y;
    endif;
?>
```



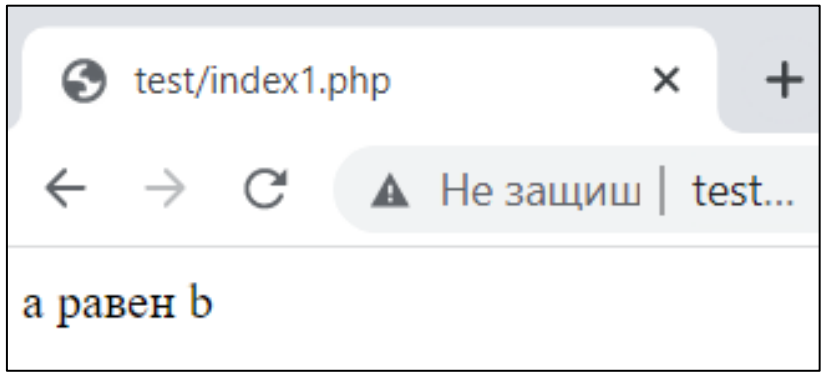
Условные операторы. Конструкция elseif

```
if (логическое_выражение_1)
   оператор_1;
elseif (логическое_выражение_2)
   оператор_2;
else
   оператор_3;
```

```
if (логическое_выражение):
   команды;
elseif(другое_логическое_выражение):
   другие_команды;
else:
   иначе_команды;
endif
```

Условные операторы. Конструкция elseif

```php
<?php
    $a = 5;
    $b = 5;
    if ($a > $b) {
        echo "a больше, чем b";
    } elseif ($a == $b) {
        echo "a равен b";
    } else {
        echo "a меньше, чем b";
    }
?>
```



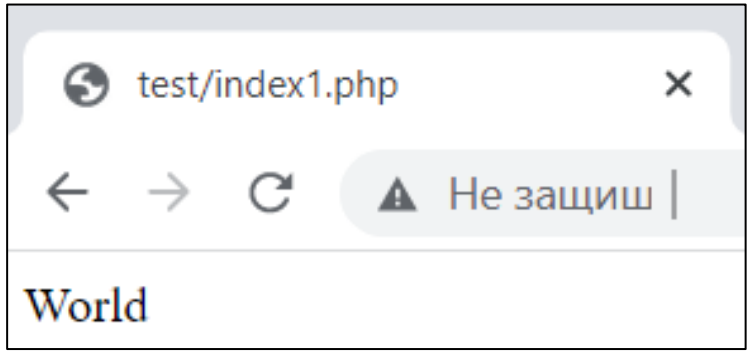
Условные операторы. Сокращенная запись if-else. Тернарный оператор

Тернарный оператор, т.е. с тремя операндами, имеет достаточно простой синтаксис, в котором слева от знака "**?**" записывается условие, а справа — два оператора разделенные знаком "**:**", слева от знака выполняется оператор при истинности условия, а справа от знака "**:**" выполняется оператор при ложном условии.

```
условие ? оператор1 : оператор2;
```

Условные операторы. Сокращенная запись if-else. Тернарный оператор

```php
<?php
$condition = "0";
$result = $condition ? 'Hello' : 'World';
echo $result;
?>
```



Если переменная **$condition** возвращает **true**, тогда переменной **$result** будет присвоен левый операнд (по левую сторону от двоеточия, т.е., **Hello**). Если условие ложное, тогда будет использоваться правый операнд (по правую сторону от двоеточия, т.е., **World**).

Конструкция выбора switch-case

Часто вместо нескольких расположенных подряд инструкций if-else целесообразно воспользоваться специальной конструкцией выбора **switch-case**. Данная конструкция предназначена для выбора действий в зависимости от значения указанного выражения. Конструкция **switch-case** чем-то напоминает конструкцию if-else, который, по сути, является ее аналогом. **Конструкцию выбора можно использовать, если предполагаемых вариантов много, например, более 5, и для каждого варианта нужно выполнить специфические действия**. В таком случае, использование конструкции if-else становится действительно неудобным.

```
switch(выражение) {
   case значение1: команды1; [break;]
   case значение2: команды2; [break;]
   . . .
   case значениеN: командыN; [break;]
   [default: команды_по_умолчанию;
   [break]]
}
```

Конструкция выбора switch-case

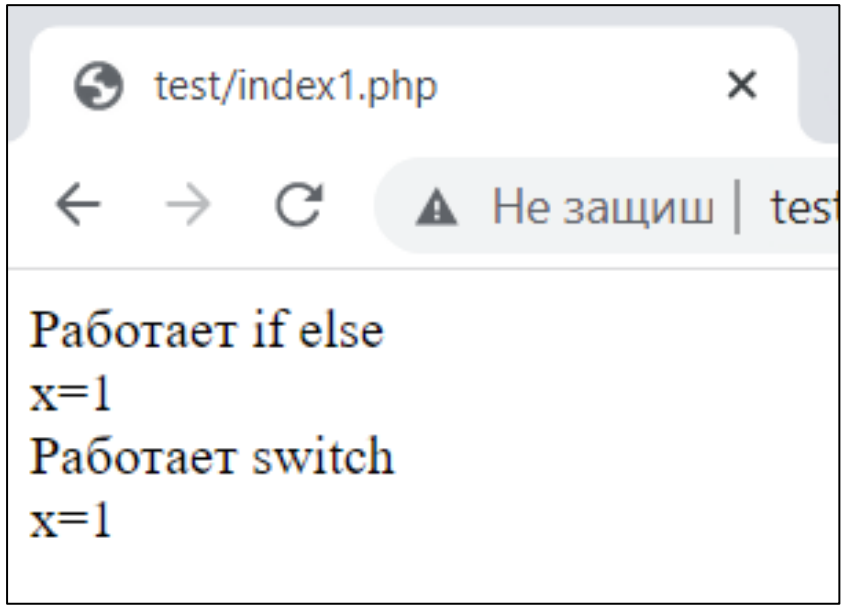
Принцип работы конструкции switch-case такой:
1. Вычисляется значение выражения;

2. Просматривается набор значений. Пусть значение1 равно значению выражения, вычисленного на первом шаге. Если не указана конструкция (оператор) break, то будут выполнены команды i, i+1, i+2, ... , N. В противном случае (есть break) будет выполнена только команда с номером i.

3. Если ни одно значение из набора не совпало со значением выражения, тогда выполняется блок default, если он указан.

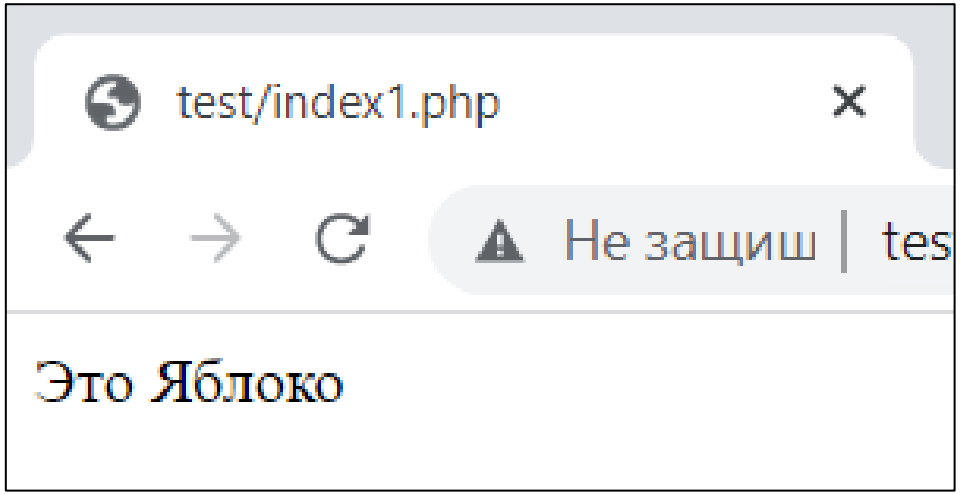
```php
<?php
    $x=1;
    echo "Работает if else", "<br>";
    if ($x == 0) {
        echo "x=0<br>";
    } elseif ($x == 1) {
        echo "x=1<br>";
    } elseif ($x == 2) {
        echo "x=2<br>";}

    echo "Работает switch", "<br>";
    switch ($x) {
        case 0:
            echo "x=0<br>";
            break;
        case 1:
            echo "x=1<br>";
            break;
        case 2:
            echo "x=2<br>";
            break;}
?>
```



Конструкция выбора switch-case

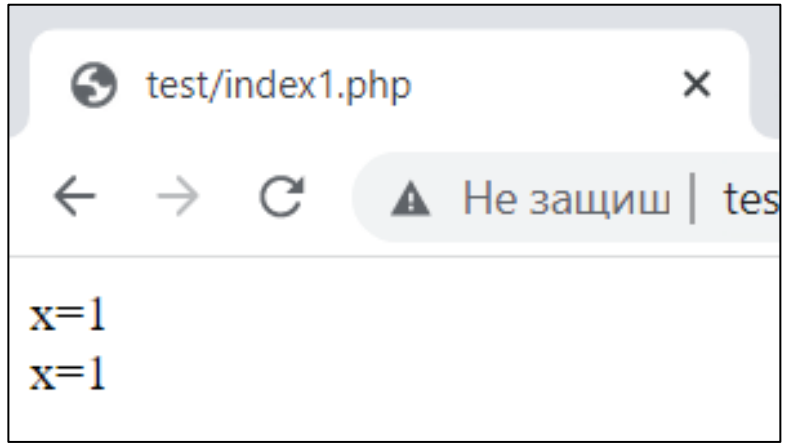
```php
<?php
    $x="Яблоко";
    switch ($x) {
    case "Яблоко":
        echo "Это Яблоко";
        break;
    case "Груша":
        echo "Это Груша";
        break;
    case "Арбуз":
        echo "Это Арбуз";
        break;
    }
?>
```



Конструкция выбора switch-case

Конструкция switch выполняется поэтапно. Сперва никакой код не исполнен. Только, когда конструкция case найдена со значением, которое соответствует значению выражения switch, PHP начинает исполнять конструкции. PHP продолжает исполнять конструкции до конца блока switch, пока не встречается оператор break. Если не использовать конструкции (операторы) break, скрипт будет выглядеть так:

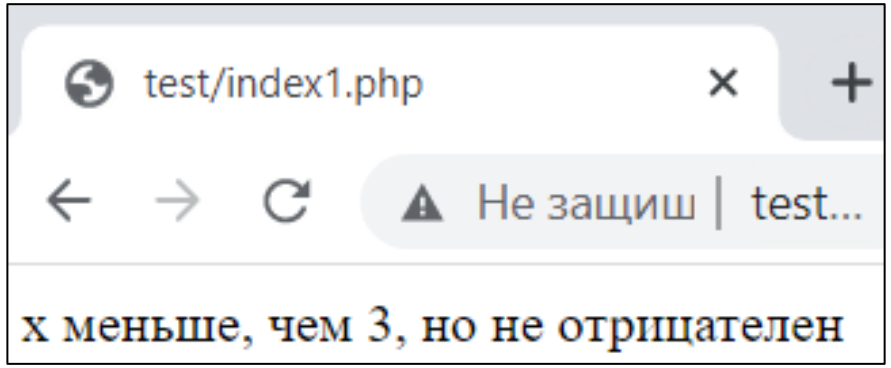
```php
<?php
    $x=1;
    switch ($x) {
        case 0:
            echo "x=$x<br>";
        case 1:
            echo " x=$x<br>";
        case 2:
            echo " x=$x<br>";
    }
?>
```



Конструкция выбора switch-case

Операторный список для **case** может быть также пуст, он просто передает управление в операторный список до следующей конструкции **case**:

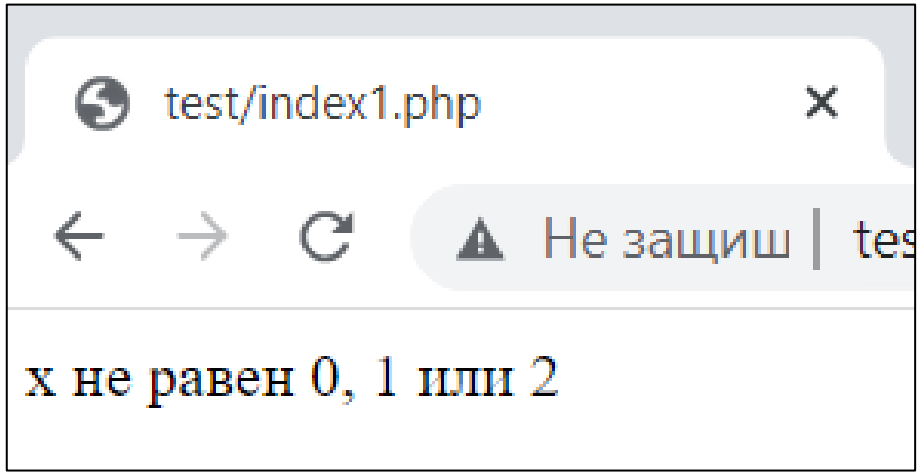
```php
<?php
$x = 2;
switch ($x) {
case 0:
case 1:
case 2:
      echo "x меньше, чем 3, но не отрицателен";
      break;
case 3:
      echo "x=3";
}
?>
```



Конструкция выбора switch-case

Когда ни одно значение из набора не совпало со значением выражения, тогда выполняется блок default, если он указан, например:

```php
<?php
$x = 3;
switch ($x) {
case 0:
      echo "x=0";
      break;
case 1:
      echo "x=1";
      break;
case 2:
      echo "x=2";
      break;
default:
      echo "x не равен 0, 1 или 2";
}
?>
```



Конструкция выбора switch-case (альтернативный вариант)

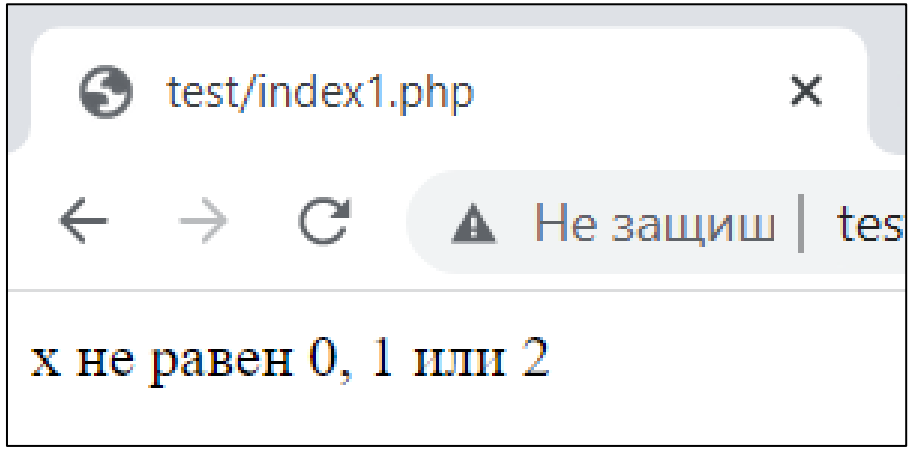
```
switch(выражение):
   case значение1: команды1; [break;]
   . . .
   case значениеN: командыN; [break;]

   [default: команды_по_умолчанию; [break];]

endswitch;
```

Конструкция выбора switch-case (альтернативный вариант)

```php
<?php
    $x=3;
    switch ($x):
    case 0:
        echo "x=0";
        break;
    case 1:
        echo "x=1";
        break;
    case 2:
        echo "x=2";
        break;
    default:
        echo "x не равен 0, 1 или 2";
    endswitch;
?>
```



Циклы в PHP

Циклы позволяют повторять определенное (и даже неопределенное - когда работа цикла зависит от условия) количество раз различные операторы. Данные операторы называются телом цикла. Проход цикла называется итерацией.

PHP поддерживает следующие виды циклов:

- Цикл с предусловием (while);
- Цикл с постусловием (do-while);
- Цикл со счетчиком (for);
- Специальный цикл перебора массивов (foreach).

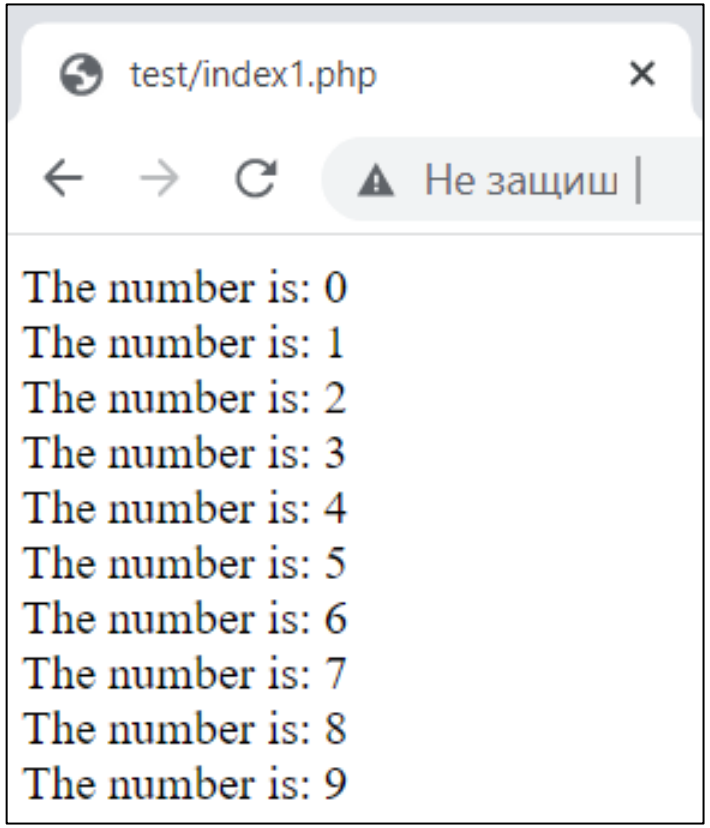
Циклы в PHP.
Цикл с предусловием while

Цикл с предусловием while работает по следующим принципам:
1. Вычисляется значение логического выражения.
2. Если значение истинно, выполняется тело цикла, в противном случае - переходим на следующий за циклом оператор.

```
while (логическое_выражение)
инструкция;
```

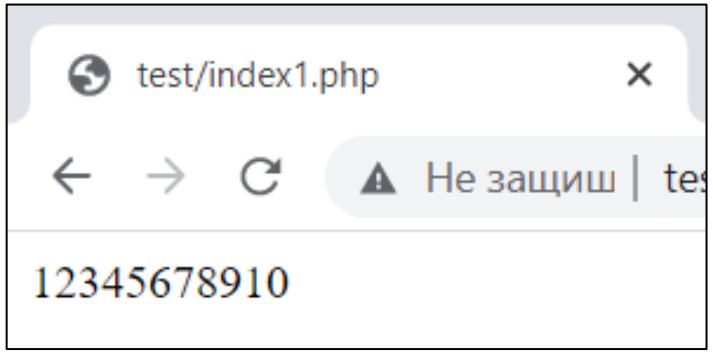
Циклы в PHP.
Цикл с предусловием while

```php
<?php
$a = 0;
while($a <= 9) {
  echo "The number is: $a <br>";
  $a++;
}
?>
```



Циклы в PHP.
Цикл с предусловием while

```php
<?php
$x=0;
while ($x++<10) echo $x;
// Выводит 12345678910
?>
```

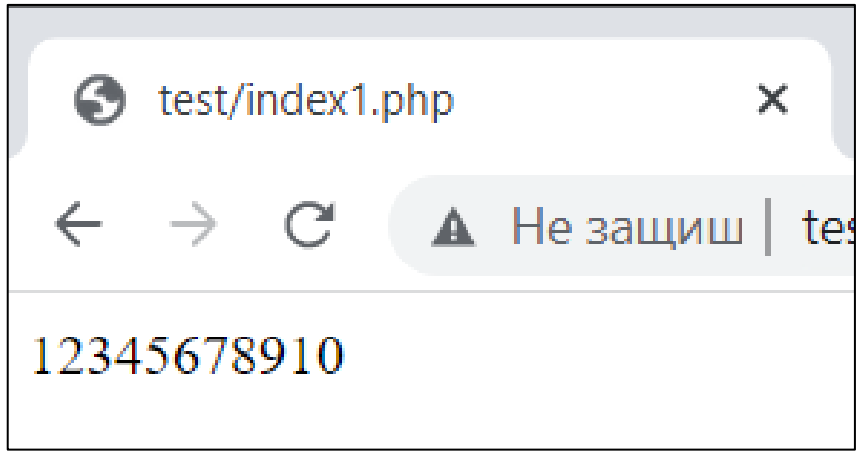


Обратите внимание на последовательность выполнения операций условия ***$x++<10***. Сначала проверяется условие, а только потом увеличивается значение переменной. Если мы поставим операцию инкремента перед переменной (++$x<10), то сначала будет выполнено увеличение переменной, а только затем - сравнение. В результате мы получили бы строку **123456789**.

Циклы в PHP.
Цикл с предусловием while

Этот же цикл можно было бы записать по-другому:

```php
<?php
    $x=0;
    while ($x<10)
    {
        $x++; // Увеличение счетчика
        echo $x;
    }
    // Выводит 12345678910
?>
```

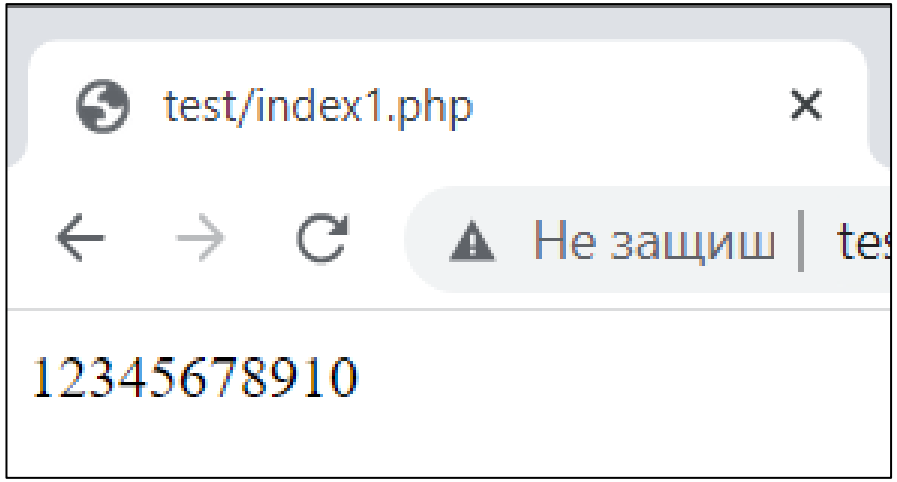


Если мы увеличим счетчик после выполнения оператора **echo**, мы получим строку *0123456789*.
В любом случае, мы имеем *10* итераций. Итерация - это выполнение операторов внутри тела цикла.

Циклы в PHP. Цикл с предусловием while (альтернативная форма записи)

```
while (логическое_выражение):
инструкция;
...
endwhile;
```

```php
<?php
    $x = 1;
    while ($x <= 10):
          echo $x;
          $x++;
endwhile;
?>
```



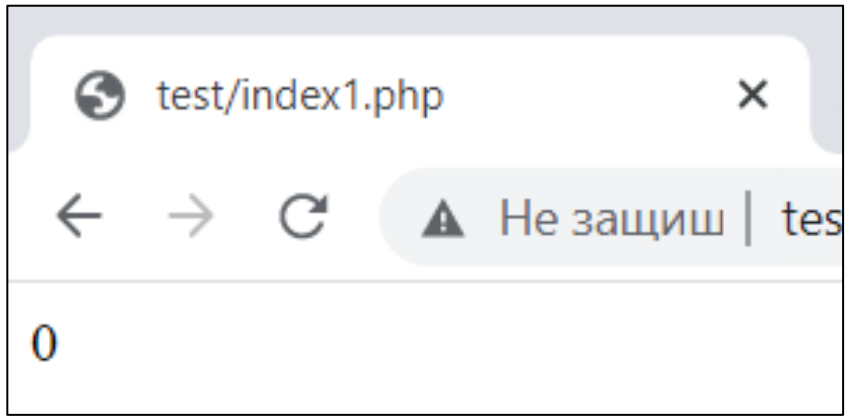
Циклы в PHP. Цикл с постусловием do while

В отличие от цикла while, этот цикл проверяет значение выражения не до, а после каждого прохода (итерации). Таким образом, тело цикла выполняется хотя бы один раз.

```
do
{
тело_цикла;
}
while (логическое_выражение);
```

Циклы в PHP. Цикл с постусловием do while

```php
<?php
$i = 0;
do {
    echo $i;
} while ($i > 0);
?>
```



В примере цикл будет выполнен ровно один раз, так как после первой итерации, когда проверяется истинность выражения, она будет вычислена как **FALSE** (*$i* не больше 0) и выполнение цикла прекратится.

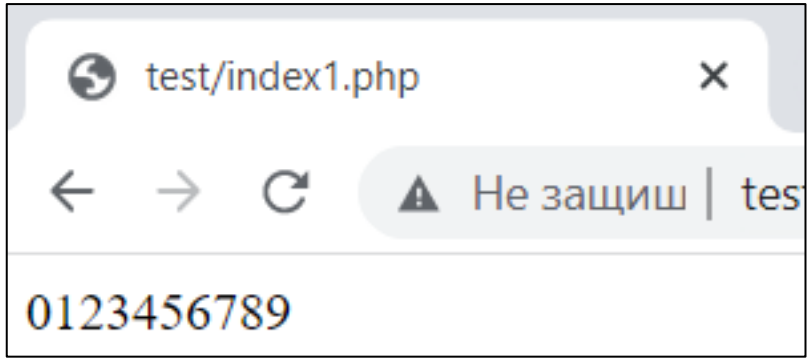
Циклы в PHP. Цикл со счетчиком for

Цикл со счетчиком используется для выполнения тела цикла определенное число раз. С помощью цикла **for** можно (и нужно) создавать конструкции, которые будут выполнять действия совсем не такие тривиальные, как простая переборка значения счетчика.

```
for (инициализирующие_команды; условие_цикла; команды_после_итерации)
{ тело_цикла; }
```

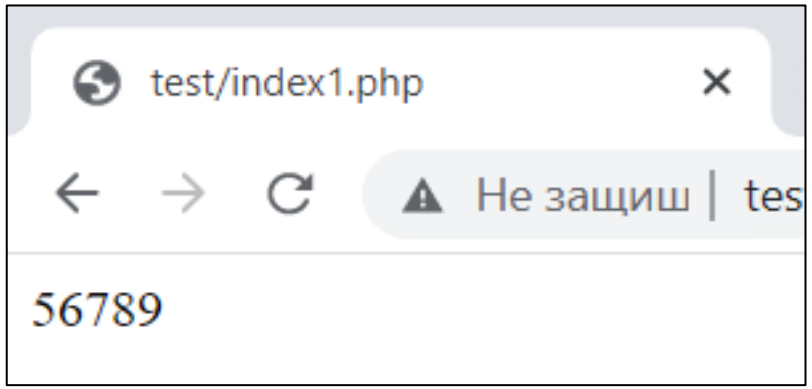
Циклы в PHP. Цикл со счетчиком for

```php
<?php
  for ($x=0; $x<10; $x++) echo $x;
?>
```



Объявление цикла for может опускать отдельный части. Например, опустить определение счетчика (он может быть определен вне цикла):

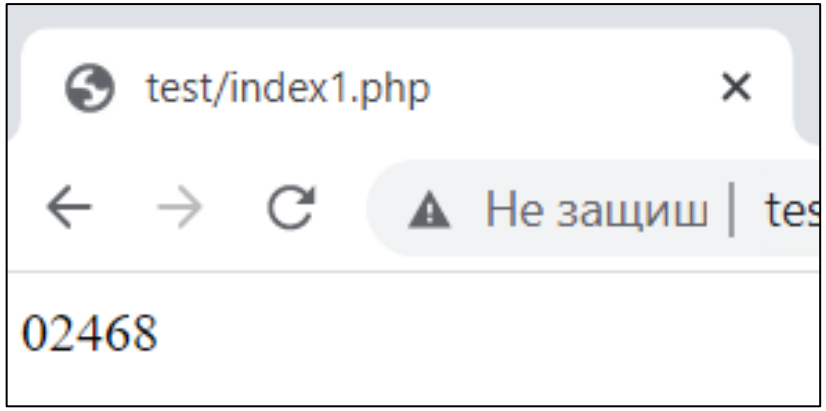
```php
<?php
$i = 5;
for (; $i < 10; $i++) echo $i;
?>
```



Циклы в PHP. Цикл со счетчиком for

Можно опустить изменение значения счетчика и изменять его внутри цикла:

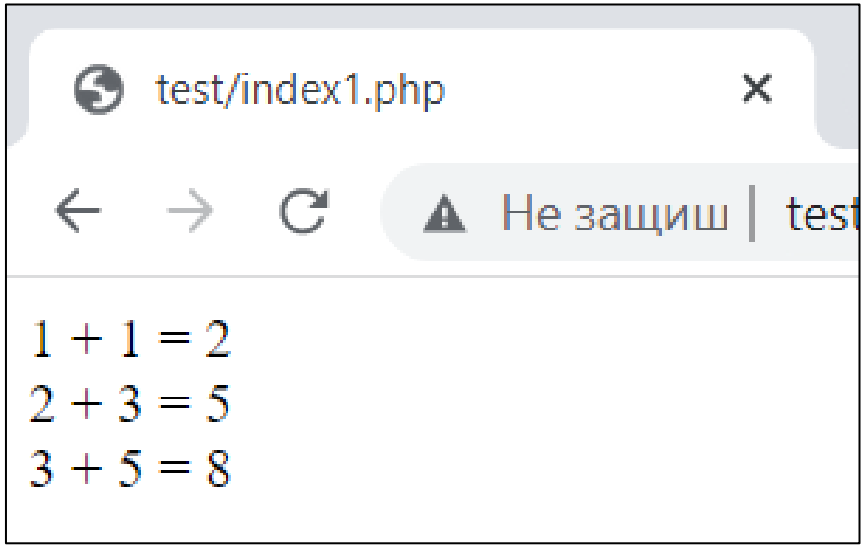
```php
<?php
$i = 0;
for (; $i < 10;)
{
    echo $i;
    $i += 2;
}
?>
```



Циклы в PHP. Цикл со счетчиком for

Можно в объявлении цикла определять и использовать сразу несколько переменных:

```php
<?php
for ($i=1, $j=1; $i + $j < 10; $i++, $j+=2)
{
    echo "$i + $j = " . $i + $j . "<br>";
?>
```

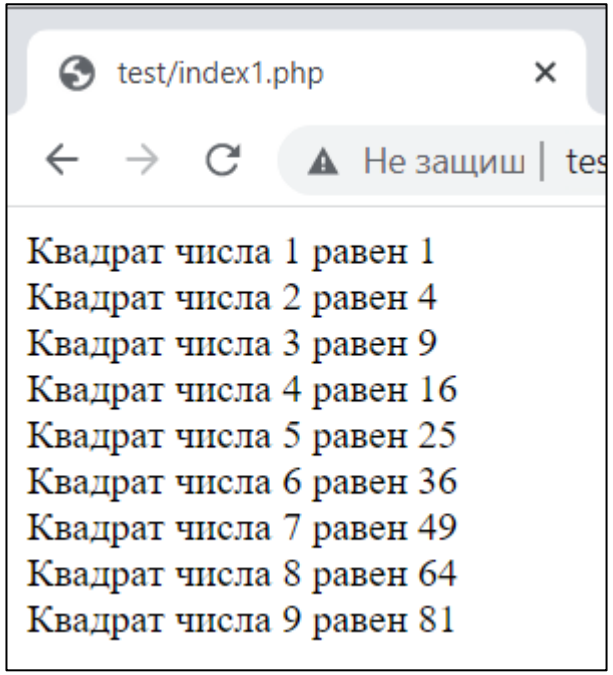


В данном случае в объявлении цикла определяются две переменных: **$i** и **$j**. При каждой итерации переменная **$i** увеличивается на 1, а **$j** - на 2. При этом цикл продолжается, пока сумма двух переменных не достигнет 10.

Циклы в PHP. Цикл со счетчиком for (альтернативный синтаксис)

```
for(инициализирующие_команды; условие_цикла; команды_после_итерации):
операторы;
endfor;
```

```
<?php
for ($i = 1; $i < 10; $i++):
    echo "Квадрат числа $i равен " .
$i * $i . "<br/>";
endfor;
?>
```

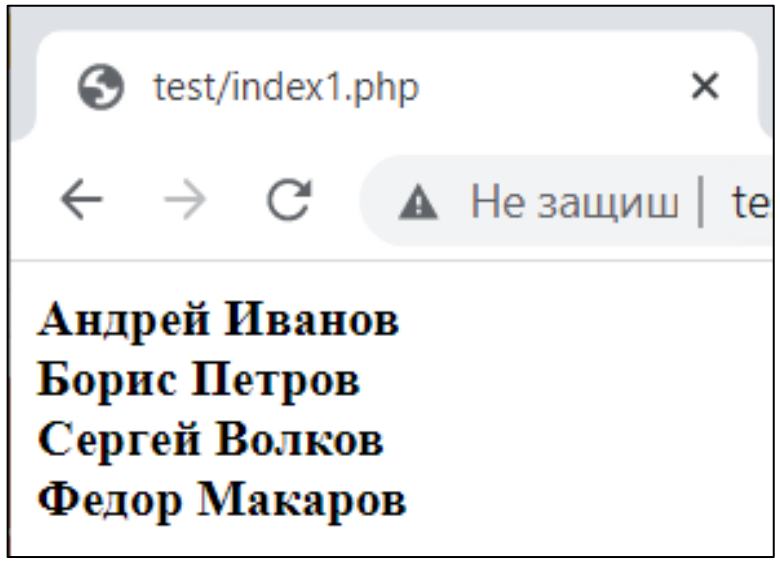


Циклы в PHP.
Цикл перебора массивов foreach

Данный цикл предназначен специально для перебора массивов. Синтаксис цикла foreach выглядит следующим образом.

```
foreach (массив as $ключ=>$значение)
команды;
```

```php
<?php
    $names["Иванов"] = "Андрей";
    $names["Петров"] = "Борис";
    $names["Волков"] = "Сергей";
    $names["Макаров"] = "Федор";
    foreach ($names as $key => $value) {
    echo "<b>$value $key</b><br>";
    }
?>
```



Циклы в PHP.
Цикл перебора массивов foreach

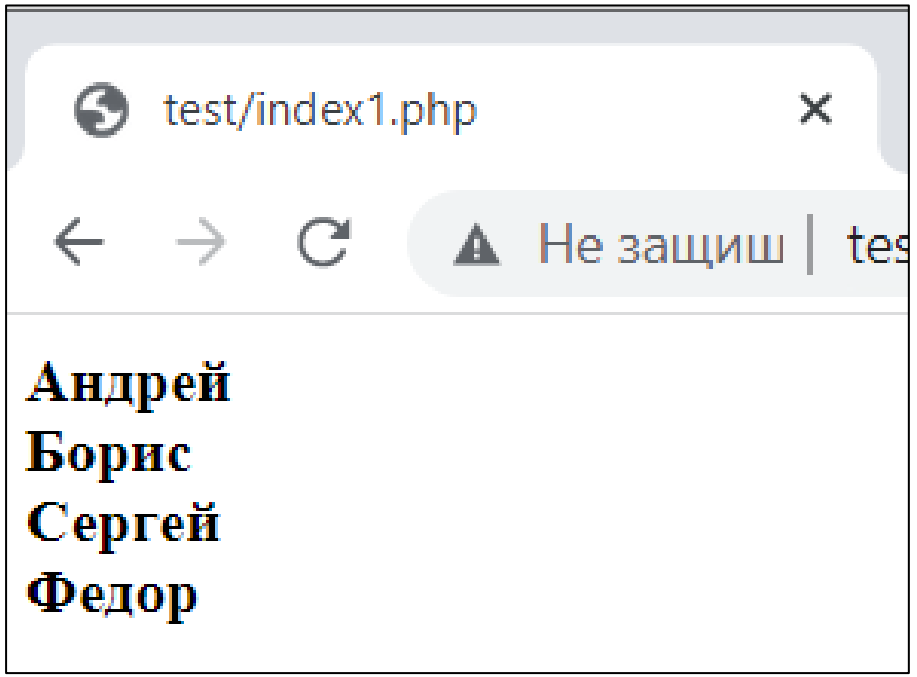
У цикла foreach имеется и другая форма записи, которую следует применять, когда нас не интересует значение ключа очередного элемента. Выглядит она так:

```
foreach (массив as $значение)
команды;
```

Важно: цикл foreach оперирует не исходным массивом, а его копией. Это означает, что любые изменения, которые вносятся в массив, не могут быть "видны" из тела цикла. Что позволяет, например, в качестве массива использовать не только переменную, но и результат работы какой-нибудь функции, возвращающей массив (в этом случае функция будет вызвана всего один раз - до начала цикла, а затем работа будет производиться с копией возвращенного значения).

Циклы в PHP.
Цикл перебора массивов foreach

```php
<?php
$names[] = "Андрей";
$names[] = "Борис";
$names[] = "Сергей";
$names[] = "Федор";
foreach ($names as $value)
{
echo "<b>$value</b><br>";
}
?>
```



Циклы в PHP.
Конструкция break

Очень часто для того, чтобы упростить логику какого-нибудь сложного цикла, удобно иметь возможность его прервать в ходе очередной итерации (к примеру, при выполнении какого-нибудь особенного условия). Для этого и существует конструкция **break**, которая осуществляет немедленный выход из цикла. Она может задаваться с одним необязательным параметром - числом, которое указывает, из какого вложенного цикла должен быть произведен выход. По умолчанию используется **1**, т. е. выход из текущего цикла, но иногда применяются и другие значения. Синтаксис конструкции **break**:

```
break; // По умолчанию
break(номер_цикла);
// Для вложенных циклов (указывается номер прерываемого цикла
```

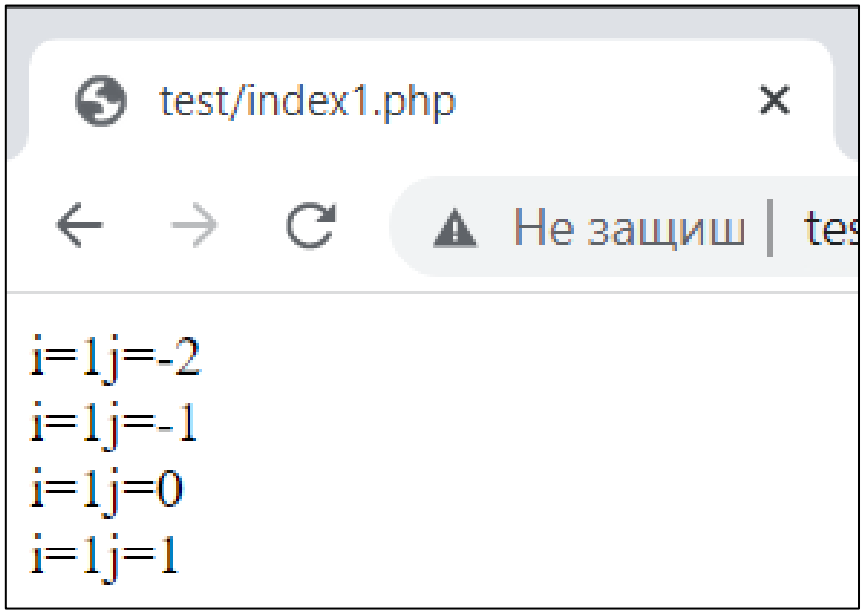
Циклы в PHP.
Конструкция break

Если нам нужно прервать работу определенного (вложенного) цикла, то нужно передать конструкции break параметр - номер_цикла, например, break(1). Нумерация циклов выглядит следующим образом:

```
for (...) // Третий цикл
{
  for (...) // Второй цикл
  {
     for (...) // Первый цикл
     {
     }
  }
}
```

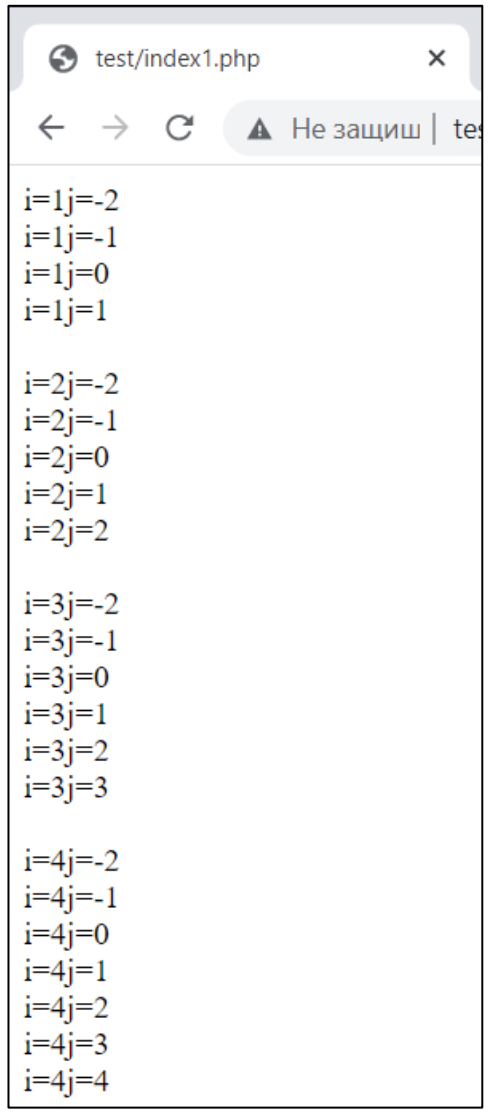
Циклы в PHP.
Конструкция break

```php
<?php
    for ($i=1; $i<=4; $i++){
        for ($j=-2; $j<=15; $j++){
            echo "i={$i}","j=$j","<br>";
            if ($j>=$i) break(2);
        }
    echo "<br>";
    }
?>
```



Циклы в PHP.
Конструкция break

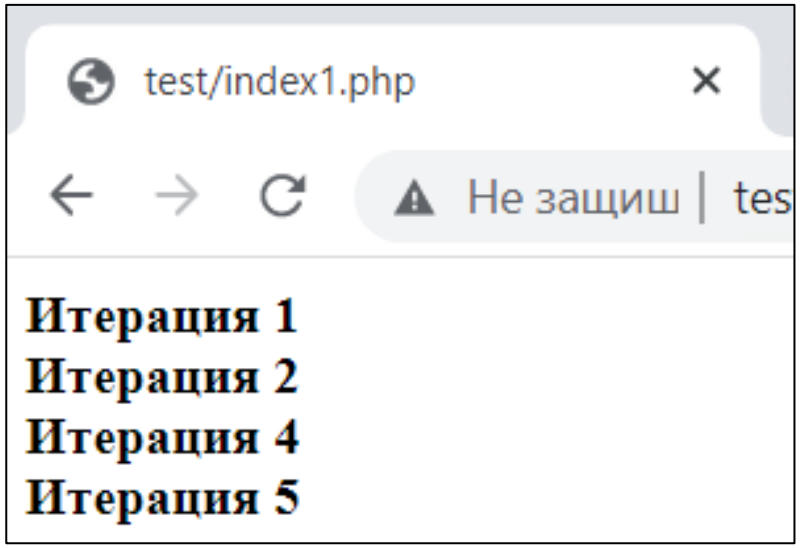
```php
<?php
    for ($i=1; $i<=4; $i++){
        for ($j=-2; $j<=15; $j++){
            echo "i={$i}","j=$j","<br>";
            if ($j>=$i) break(1);
        }
    echo "<br>";
    }
?>
```



Циклы в PHP.
Конструкция continue

Конструкция continue так же, как и break, работает как правило "в паре" с циклическими конструкциями. Она немедленно завершает текущую итерацию цикла и переходит к новой (конечно, если выполняется условие цикла для цикла с предусловием). Точно так же, как и для break, для continue можно указать уровень вложенности цикла, который будет продолжен по возврату управления.

```php
<?php
$x=0;
while ($x++<5) {
if ($x==3) continue;
echo "<b>Итерация $x</b><br>";
      }
// Цикл прервется только на третьей итерации
?>
```



Конструкции возврата значений

Конструкция return

1. Конструкция return возвращает значения, преимущественно из пользовательских функций, как параметры функционального запроса. При вызове return исполнение пользовательской функции прерывается, а конструкция return возвращает определенные значения.

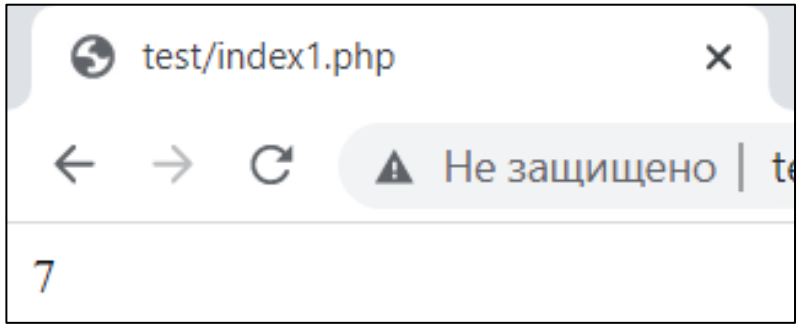
2. Если конструкция return будет вызвана из глобальной области определения (вне пользовательских функций), то скрипт также завершит свою работу, а return также возвратит определенные значения.

3. Возвращаемые значения могут быть любого типа, в том числе это могут быть массивы и объекты. Возврат приводит к завершению выполнения функции и передаче управления обратно к той строке кода, в которой данная функция была вызвана.

Конструкции возврата значений

Конструкция return

```php
<?php
function retfunct()
{
     return 7;
}
echo retfunct(); // выводит '7'
?>
```




```php
<?php
function numbers()
{
  return array (0, 1, 2);
}
list ($zero, $one, $two) = numbers();
echo $zero, "<br>";
echo $one, "<br>";
?>
```

